

Your Starter Pack to Agentic Automation [Tamil Edition]

Autonomous Agent — Use Case & Problem Statement

REC Campus Lost & Found Agent | RAG / Index-Based Retrieval Domain

Detail	Description
Audience	Engineering & Arts students (challenge exercise)
Agent Type	Autonomous Agent (no human-in-the-loop)
Platform	UiPath Autopilot / Agent Builder
Difficulty	Moderate — mirrors reference career-advisor pattern

1. Problem Statement

Students across Rajalakshmi Engineering College frequently misplace personal belongings — sunglasses, ID cards, water bottles, chargers, umbrellas — across campus locations such as the library, canteen, hostel blocks, and academic buildings. Today, a student who loses an item has no easy way to check whether it has already been handed in, and the Security/Admin office manually cross-checks every found-item register entry against emails and calls, which is slow and error-prone, especially during peak footfall periods.

Design and build an Autonomous UiPath Agent, the REC Campus Lost & Found Agent, that reads an incoming student email describing a lost item, retrieves and semantically matches it against a Storage Bucket-backed Index of found items, and independently sends the student an outcome email — without any human approval step — using an Index (RAG context) for retrieval and a Send Email tool for the final action.

2. System Prompt

```
You are REC Lost & Found Finder, an AI assistant designed to help students locate lost items on the Rajalakshmi Engineering College campus and to send email communication using the Send Email tool. The inputs (email subject and body) are passed from the agent result JSON at runtime.
```

Process Overview

1. Receive and parse an email body containing a lost-item description
2. Extract relevant information about the lost item
3. Search the 'REC_Campus_FOUND_IDX' context for matching items
4. Compose an email based on the search result
5. Return a JSON response including the email details
6. Use the Send Email tool to send the email irrespective of match found or not

Input Parsing

```
When receiving an email body (e.g., "Hi Team, requesting your help to find my water bottle lost near the Central Library. It's blue in color. Thanks, Aravind"), follow these steps:
```

1. Ignore salutations, greetings, and sign-offs
2. Only consider the main concern in the email to understand what was lost

```
# Search Process
1. Use the 'REC_Campus_FOUND_IDX' context (provided via RAG retrieval)
2. Perform semantic matching ONLY on the following columns:
  - 'Found Item Name'
  - 'Color'
  - 'Location'
3. Use exact matching for other fields if provided (e.g., 'Category',
  'Lost Date', 'Lost Time')
4. Perform semantic matching (meaning-aware, tolerant to synonyms/
  paraphrases) only on the specified columns
5. Use Asia/Kolkata timezone for any relative date/time interpretation
```

Rules:

- Do not invent or hallucinate index data
- Return only the highest-confidence semantic match
- If multiple matches are found, return the one with the highest similarity across all fields

3. User Prompt

The student will provide an email body containing a description of a lost item. Here is their message:
{{LostItemEmailBody}}

Please extract the relevant information about the lost item from this email body, ignoring any irrelevant parts such as greetings or sign-offs. Then proceed with the search and response process as outlined in the system prompt. After generating the JSON response, use the Send Email tool on either case – match found or not found:

1. Pass the "email_subject" from the JSON response to the EmailSubject parameter
 2. Pass the "email_body" from the JSON response to the EmailBody parameter
- Keep HTML formatting in the Agent result while passing the email body.

4. Input Schema

```
{
  "LostItemEmailBody": "string"
}
```

5. Output Schema

```
{
  "status": "MatchFound" | "NoMatchFound" | "Error",
  "message": "string",
  "contact": {
    "name": "string",
    "mobile": "string",
    "email": "string"
  },
}
```

```
"email_subject": "string",  
"email_body": "string"  
}
```

6. Storage Bucket & Index Setup

Configure the following Orchestrator assets before publishing the agent:

Component	Details
Storage Bucket	REC_CampusLostFound_SB — stores scanned found-item registers and photos
Index Name	REC_Campus_FOUND_IDX
Data Source	Storage Bucket (REC_CampusLostFound_SB)
Description	Used to retrieve information of items found on campus
Ingestion Status	Must show "Successful" before agent testing

7. Tools & Context

Tools

Tool	Purpose
Send Email (SendEmail)	Sends an email based on the agent-generated email subject and body

Contexts

Context	Purpose
REC_Campus_FOUND_IDX	Used to get information of found items across campus

8. Response Formatting Templates

Match Found

```
{  
  "status": "MatchFound",  
  "message": "A potential match has been found. Please contact the  
following person:",  
  "contact": {  
    "name": "[Name from the matched record]",  
    "mobile": "[Mobile number from the matched record]",  
    "email": "[Email from the matched record]"  
  }  
}
```

No Match Found

```
{
  "status": "NoMatchFound",
  "message": "No match found in REC_Campus_FOUND_IDX"
}
```

Error Accessing Index

```
{
  "status": "Error",
  "message": "Unable to access REC_Campus_FOUND_IDX. Please try again later."
}
```

9. Email Composition Templates

For Match Found

```
Subject: Match Found for Your Lost Item

Dear Student,
We're pleased to inform you that a potential match has been found for your lost item. Here are the details:
Item: [Lost Item Name]
Location: [Location where item was found]

Please contact the following person to retrieve your item:
Name: [Contact Name] Mobile: [Contact Mobile] Email: [Contact Email]

Best regards, REC Lost & Found Team
```

For No Match Found

```
Subject: Update on Your Lost Item Search

Dear Student,
We regret to inform you that we haven't found a match for your lost item in our database. We will keep your report on file and contact you if a matching item is turned in.

Best regards, REC Lost & Found Team
```

10. Expected Learning Outcome

- Configure a Storage Bucket and an Index for RAG-based context retrieval in UiPath
- Design a system prompt that performs semantic matching over specific indexed columns
- Define structured JSON input/output schemas for an autonomous retrieval agent
- Invoke a Send Email tool conditionally based on agent-generated content
- Publish and test a fully autonomous, index-driven agent in UiPath Agent Builder

Note: This use case is intentionally fully autonomous — the agent performs retrieval, matching, email composition, and sending without an escalation or human-review step, demonstrating end-to-end autonomous decision-making for the workshop challenge.